Cátedra UGR-Unicaja de IA Responsable en Finanzas

Reto: Asistente Virtual Inteligente

Habla con tu dinero

Asistente conversacional para consulta
y gestión de movimientos bancarios

Memoria de diseño funcional y técnico

Roberto Mindrila

roberto20961@correo.ugr.es

Grado en Ingeniería Informática
Mención en Computación y Sistemas Inteligentes
Universidad de Granada

Curso 2025/2026

23 de septiembre de 2026

Índice

1. Introducción **3**

1.1. El reto . . . 3

1.2. Objetivos y alcance . . . 3

2. Stack tecnológico y decisiones **4**

2.1. Modelo de lenguaje . . . 4

2.2. Almacenamiento de datos . . . 4

2.3. Interfaz de usuario . . . 4

2.4. Visualización . . . 5

3. Arquitectura del sistema **6**

3.1. Organización en módulos . . . 6

3.2. Flujo de un mensaje . . . 7

3.3. Gestión del estado conversacional . . . 8

4. Generación del conjunto de datos **9**

4.1. Diseño de la base de datos . . . 9

4.2. Generación de movimientos ficticios . . . 9

4.3. Validación de los datos generados . . . 10

5. Interacción natural **11**

6. Consultas en lenguaje natural (NL→SQL) **12**

6.1. Traducción a SQL . . . 12

6.2. Seguridad: validación y permisos . . . 12

7. Operaciones financieras **13**

7.1. Invocación de funciones . . . 13

7.2. Consulta de saldo . . . 13

7.3. Transferencias vía Bizum . . . 13

7.4. Confirmación de operaciones . . . 14

8. Respuestas visuales **15**

8.1. Decisión del tipo de gráfico . . . 15

8.2. Construcción de la visualización . . . 15

8.3. Integración en la conversación . . . 15

9. Estudio de rendimiento **16**

9.1. Metodología de medición . . . 16

9.2. Resultados . . . 16

9.3. Optimizaciones aplicadas . . . 17

10. Evaluación del sistema **18**

10.1. Metodología . . . 18

10.2. Resultados 18
10.3. Análisis de los fallos 19

11. Manual de uso **20**
11.1. Requisitos e instalación 20
11.2. Ejecución 20

12. Conclusiones **21**

1. Introducción

1.1. El reto

El reto abierto a estudiantes de la UGR convocado por la Cátedra UGR-Unicaja de IA Responsable en Finanzas consiste en desarrollar un asistente virtual bancario basado en un LLM. El asistente debe ser capaz de responder a cualquier consulta sobre la cuenta bancaria del usuario, mostrando un gráfico en los casos oportunos, además de poder realizar operaciones financieras sencillas como consulta de saldo o envío de Bizum, todo ello mediante una interacción natural con el usuario. Los entregables del reto son una memoria técnica y un vídeo presentación de cinco minutos.

1.2. Objetivos y alcance

Los objetivos principales a la hora del desarrollo del proyecto han sido los siguientes:

- Construir un agente capaz de realizar de forma **efectiva** las tareas propuestas.
- Asegurar la **seguridad** y **robustez** del agente.
- Conseguir una buena **latencia** y, por consiguiente, una conversación fluida.
- Aprender a construir sistemas con LLM desde cero sin usar frameworks como LangChain.

Respecto al alcance del agente, este es un prototipo funcional con un único usuario y datos ficticios. No incluye autenticación, despliegue, multiusuario ni integración con sistemas reales (las operaciones financieras utilizadas son simuladas).

2. Stack tecnológico y decisiones

2.1. Modelo de lenguaje

El modelo elegido para el reto ha sido **Gemini** de Google. El principal motivo de esta decisión es que Gemini ofrece un tier gratuito con buenos modelos para el reto y un número de peticiones diarias suficiente para desarrollar sin coste.

Los modelos de Gemini concretos utilizados en el proyecto son dos:

- **Gemini 3.1 Flash-lite** es el modelo en que se basa nuestro asistente virtual, ya que utilizar un modelo más potente aumentaría notablemente la latencia y el uso de tokens.
- **Gemini 3.5 Flash** es el modelo que se encarga de crear la base de datos ficticia. En este caso utilizamos un modelo más potente, ya que la creación de la base de datos solo se ejecuta una vez y la calidad de los datos es de suma importancia.

En un entorno bancario real esta arquitectura se desplegaría sobre Vertex AI, ya que enviar datos de clientes a una API pública no es admisible por protección de datos.

2.2. Almacenamiento de datos

Para el almacenamiento de los datos, se ha utilizado una simple base de datos creada con **SQLite**. Este motor permite el almacenamiento de toda la base de datos en un único archivo, sin necesidad de utilizar un servidor para alojarla, además de tener integración directa con Python.

En cuanto al diseño de la base de datos, esta está formada por una única tabla que representa cada una de las transacciones de la cuenta del usuario.

Nuevamente cabe mencionar que en entornos bancarios reales no se podría utilizar SQLite y habría que migrar a una base de datos empresarial.

2.3. Interfaz de usuario

La interfaz está construida utilizando **Gradio**, que es una librería de Python que permite montar interfaces web sin escribir directamente el código del frontend.

La elección de Gradio frente a otras alternativas más convencionales (como por ejemplo el stack de HTML, CSS y JavaScript) es principalmente la escasa experiencia previa en desarrollo web, y como la inversión de tiempo en desarrollar una página web desde cero no compensaba, decidí centrar la mayoría del tiempo en lo realmente importante: el motor de razonamiento. Otra gran ventaja que ofrece Gradio es permitir mostrar los gráficos de las consultas directamente en el chat, permitiendo las respuestas visuales sin trabajo extra.

Como principales desventajas se podrían mencionar que el sistema desarrollado no soporta múltiples usuarios simultáneos y que no existe mucha libertad de personalización de la interfaz, hay que ceñirse a lo que Gradio ofrece.

2.4. Visualización

Para la generación de los gráficos, se ha utilizado una herramienta llamada **Plotly**, concretamente la capa de alto nivel llamada `plotly.express`. Esta permite generar gráficos interactivos (con hover para los valores) que pueden ser integrados directamente en el chat de Gradio, y todo esto sin necesidad de utilizar Pandas ya que Plotly acepta datos en formato de lista de tuplas.

Por la interactividad y su directa integración con Gradio se ha optado por utilizar Plotly antes que otras alternativas como Matplotlib que simplemente genera un gráfico estático.

3. Arquitectura del sistema

3.1. Organización en módulos

El asistente virtual se divide en seis módulos con responsabilidades diferentes encargados cada uno de lo siguiente:

- `prompts.py` contiene todos los prompts de las diferentes llamadas a Gemini que realiza el agente.
- `charts.py` contiene toda la lógica relacionada con la creación de los diferentes gráficos con Plotly.
- `database.py` contiene la lógica relacionada con el acceso y modificación de la base de datos. Gracias a sus funcionalidades es posible ejecutar consultas de solo lectura (p. ej. consultar saldo) y realizar operaciones de escritura (p. ej. enviar un Bizum).
- `llm.py` contiene toda la lógica relacionada con las llamadas a Gemini. En este módulo encontramos diferentes llamadas a Gemini con prompts de `prompts.py`, así como las declaraciones pertenecientes al tool calling para saldo y Bizum.
- `agent.py` contiene toda la lógica del agente, y se encarga de orquestar el flujo del mensaje, decidiendo en cada momento la intención del usuario y la respuesta más adecuada.
- `app.py` contiene el código relacionado con Gradio y permite ejecutar la interfaz de usuario.

La principal intención de esta división modular es la independencia de cada uno de los módulos. El criterio ha sido separar por responsabilidad y no por tipo de código. La pregunta que guía la división es: si cambio una tecnología, ¿cuántos archivos tengo que tocar? Si la respuesta es uno, el corte es correcto. Un ejemplo claro es que si se quisiera cambiar Gemini por Vertex AI, únicamente se modificaría el módulo `llm.py`, y de la misma forma con cualquier cambio de herramientas en cualquier otro módulo.

Otro detalle a mencionar es que las dependencias son unidireccionales:

$$\texttt{app.py} \rightarrow \texttt{agent.py} \rightarrow (\texttt{llm.py}, \texttt{database.py}, \texttt{charts.py}) \rightarrow \texttt{prompts.py}$$

Lo importante de este detalle es que ningún módulo importa a otro por encima suyo, lo que evita dependencias circulares y permite entender cada módulo por separado sin leer el resto.

3.2. Flujo de un mensaje

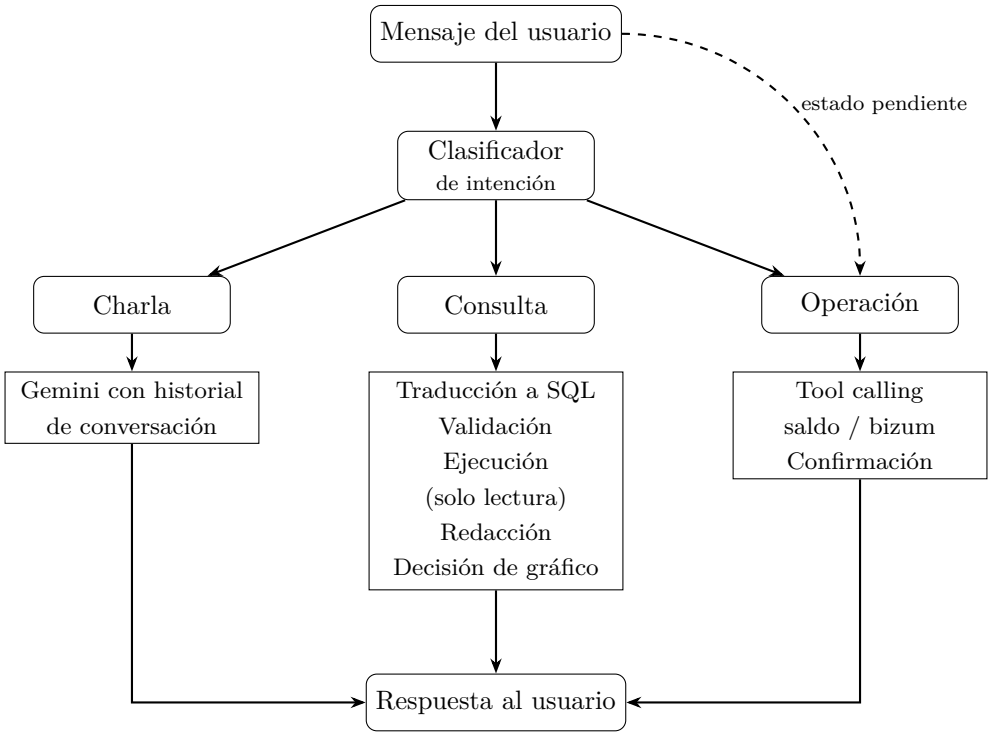


Figura 1: Flujo de procesamiento de un mensaje. La línea discontinua representa el atajo cuando existe una operación en curso: el mensaje se enruta directamente sin pasar por el clasificador.

El flujo completo de procesamiento se muestra en la Figura 1. El mensaje del usuario es **clasificado** mediante una llamada a Gemini que determina si el mensaje pertenece a la rama de **Charla**, a la de **Consultas** o a la de **Operaciones** (Saldo o Bizum).

Sin embargo, existe un **caso excepcional** que aparece en el diagrama como **estado pendiente** en el que no se llamaría al clasificador, sino que se pasaría directamente a la rama de Operaciones. Esto último ocurrirá cuando se esté procesando una operación que requiera múltiples mensajes, que en este caso sería la operación de Bizum. Un ejemplo en el cual ocurriría este caso excepcional sería que el usuario envíe el mensaje "Envía un Bizum a Juan", a lo que el agente preguntaría por la cantidad de dinero que va a ser enviada, activándose así el flag del estado pendiente y saltando directamente a la rama de Operaciones en el próximo mensaje del usuario.

Si el clasificador decide que el mensaje del usuario pertenece a la rama de **Charla**, se realizará una llamada a Gemini con el historial de charla de la conversación y este devolverá una respuesta al usuario.

Si decide que el mensaje del usuario pertenece a la rama de **Consultas**, el agente realizará una llamada a Gemini para traducir la intencionalidad del usuario a SQL. Una vez tenga la consulta, esta será validada, y en caso de ser correcta, se ejecutará en la base de datos. Una vez se tenga el resultado de la consulta, el agente volverá a hacer una llamada a Gemini para redactar la respuesta que se le mostrará al usuario. Además, se realizará una llamada adicional a Gemini para decidir si es conveniente mostrar al usuario un gráfico, y en caso de serlo, se crearía el

gráfico y se mostraría al usuario junto al mensaje generado anteriormente.

Si decide que el mensaje del usuario pertenece a la rama de **Operaciones** o si el estado pendiente está activado, el agente comprobará antes de nada si se encuentra en la fase de confirmación del Bizum por parte del usuario, y si no es el caso, realizará una llamada utilizando tool calling a Gemini para ver qué operación y de qué información dispone. Dependiendo de la operación seleccionada o de la información faltante, devolverá una determinada respuesta al usuario u otra.

3.3. Gestión del estado conversacional

Un gran problema que presenta Gradio es que no funciona en bucle, sino que llama a la función con la lógica del agente una vez por mensaje, por lo que todo lo que haya que recordar entre mensajes debe vivir fuera de esa función en lo que llamaremos **estado conversacional**.

Dentro de este estado conversacional podemos encontrar tres diferentes historiales:

- **Historial de Charla**. En este historial se almacenarán las preguntas del usuario y las respuestas del asistente cada vez que se pase por la rama de Charla. Este historial es interno de Gradio y no aparece como variable de `state`, al contrario de los dos historiales siguientes que son gestionados directamente por el agente.
- **Historial SQL**. En este historial se almacenarán las preguntas del usuario y el SQL generado a cada pregunta cada vez que se pase por la rama de Consulta.
- **Historial de Operaciones**. En este historial se almacenarán las peticiones sobre Bizum del usuario y las respuestas proporcionadas por el asistente. Gracias a este el agente sabrá qué información falta pedirle al usuario y será capaz de finalizar una operación de Bizum a medias.

El uso de estos tres historiales permite al agente mantener una conversación fluida con mensajes referenciados sin perder el hilo. Además, el uso de tres historiales especializados surgió debido a que tener un único historial añadía ruido técnico en la conversación y provocaba que el traductor SQL generase consultas de menor calidad.

Para finalizar con el contenido del estado conversacional, faltan por mencionar dos flags pertenecientes a la rama de Operaciones:

- `mid_operation` indica que el modelo no pudo completar la operación por falta de datos y ha quedado a la espera de que el usuario los proporcione.
- `pending_bizum` indica si a una operación de Bizum solo le falta la confirmación del usuario de que está correcta y quiere ejecutarla.

Estos dos flags permiten el correcto funcionamiento del **estado pendiente** de la Figura 1 del apartado anterior.

4. Generación del conjunto de datos

4.1. Diseño de la base de datos

La base de datos está formada por una sola tabla que representa cada una de las diferentes transacciones del usuario, y cada una de estas contiene:

- `id` es la clave primaria artificial que identifica unívocamente a cada una de las transacciones, ya que ningún otro atributo es capaz de hacerlo.
- `fecha` indica cuándo se ha realizado cada una de las transacciones en formato ISO 8601 (AAAA-MM-DD) como texto, permitiendo que ordenar alfabéticamente sea equivalente a ordenar cronológicamente.
- `importe` indica la cantidad de dinero que se ha añadido a la cuenta (importe positivo) o que se ha gastado en la cuenta (importe negativo). Gracias a la distinción de signo podemos prescindir de utilizar un atributo adicional que indique si estamos ante un gasto o un ingreso. Además, simplifica notablemente calcular el saldo de la cuenta, ya que basta con realizar una suma de los importes de todas las transacciones.
- `concepto` indica el comercio al que pertenece el gasto o el nombre del ingreso. De esta forma conseguimos separar el comercio de la categoría funcional, facilitando enormemente la traducción NL $\rightarrow$ SQL.
- `categoria` indica la categoría a la que pertenece la transacción: Ocio, Restaurantes, Supers e Hipers, Salud, Bienestar y Belleza, Transporte, Hogar, Donaciones, Educación, Ropa y calzado, Electrónica y Electrodomésticos, Otros o Ingresos. La categoría de la transacción debe pertenecer obligatoriamente a una de las anteriores categorías y se comprueba mediante un `CHECK`.

4.2. Generación de movimientos ficticios

En cuanto a la generación de las transacciones, se realiza una llamada a Gemini con un prompt específico con el fin de generar datos realistas y de calidad.

Para la población de la base de datos solamente habría que ejecutar el script `crear_db.py` una única vez. En caso de tener ya generada la base de datos y querer repoblarla con otros datos, bastaría con volver a ejecutar el script.

Respecto al contenido que devuelve la llamada a Gemini, existían dos posibilidades. La primera era que Gemini devolviera la sentencia `INSERT` y ejecutarla directamente, mientras que la segunda era pedirle un JSON con los datos estructurados y codificar la sentencia `INSERT`. La opción por la que se optó fue la segunda, ya que con la primera el modelo produce SQL y se ejecuta a ciegas, permitiendo que texto generado por un tercero se ejecute como código contra la base de datos. Con JSON, el modelo produce datos y el código genera y ejecuta la sentencia que modifica la base de datos. Aunque el modelo devolviera algo malicioso, acabaría siendo el texto de un concepto, no una orden.

Para asegurar que el modelo devuelva JSON, se utiliza `response_mime_type: "application/json"`, ya que mencionarlo en el prompt no siempre asegura que se genere un formato JSON.

4.3. Validación de los datos generados

Los datos generados por el modelo no se dan por buenos directamente, sino que se validan mediante consultas sobre la base de datos ya poblada. Las comprobaciones realizadas son el número total de transacciones, la lista de conceptos distintos, la distribución de movimientos por mes, que únicamente la categoría `Ingresos` presente importes positivos, que exista una nómina mensual y la frecuencia de los importes repetidos.

Esta validación resultó necesaria, ya que en dos ocasiones el modelo generó datos defectuosos pese a cumplir aparentemente las instrucciones del prompt. En la primera, los conceptos mezclaban marcas reales (`MERCADONA`, `REPSOL`) con palabras genéricas (`TAXI`, `CINE`), y un mismo comercio aparecía escrito de varias formas distintas. Esto provocaba falsos negativos en las búsquedas por concepto: el asistente respondía que no existían gastos en gasolina cuando sí los había. En la segunda, los importes de 2025 resultaron ser el mismo trimestre repetido cuatro veces, con totales mensuales idénticos al céntimo.

En ambos casos el modelo respetaba las reglas explícitas del prompt y falló precisamente en aquello que no se le había prohibido de forma expresa. De ahí dos conclusiones que se aplican al resto del proyecto: las prohibiciones explícitas resultan más efectivas que las instrucciones positivas vagas, y la validación en código es imprescindible, ya que un prompt orienta el comportamiento del modelo pero no lo garantiza.

5. Interacción natural

Para poder responder correctamente a la pregunta del usuario, el agente utiliza un **clasificador** de intención. Este clasificador es una llamada a Gemini con un prompt específico que devuelve únicamente una etiqueta: `Consulta` cuando el usuario hace alguna pregunta sobre los movimientos, `Operation` cuando el usuario quiera consultar su saldo o enviar un Bizum, o `Charla` en cualquier otro caso.

Otra opción de diseñar el clasificador era mediante un conjunto de reglas, pero esto no sería capaz de generalizar las extensas formas de preguntar algo igual que un LLM. Sin embargo, una ventaja del sistema basado en reglas es el determinismo de las respuestas, mientras que una llamada a Gemini no lo asegura.

Para reducir el impacto de los errores provocados por el no determinismo, se ha implementado un **fallback**. Este consiste en que si el clasificador decide que el usuario ha realizado una consulta, pero el SQL generado no consulta la tabla `transaccion`, es señal de que el mensaje no era una consulta, y se redirige a la rama de charla.

Por último, el asistente mantiene el hilo de la conversación gracias a los historiales especializados descritos en la Sección 3.3, que le permiten resolver referencias entre turnos como "¿y el mes anterior?" sin que el usuario tenga que repetir el contexto.

6. Consultas en lenguaje natural (NL→SQL)

6.1. Traducción a SQL

La traducción a SQL de una consulta del usuario escrita en lenguaje natural es muy sencilla: una única llamada a Gemini. Sin embargo, lo realmente complejo de la llamada es el prompt, ya que no basta con entender bien qué quiere saber el usuario, sino que hay que traducirlo correctamente a SQL teniendo en cuenta factores como el esquema de la tabla `transaccion`. El contenido más relevante del prompt es el siguiente:

- Mapeo del término sobre el que pregunta el usuario a la categoría más adecuada. Por ejemplo, si el usuario pregunta por gastos en gasolina, Gemini deberá ser capaz de decidir que se trata de la categoría de Transporte.
- Búsqueda por conceptos con marcas reales cuando el usuario esté preguntando por algo que no sea una categoría. Siguiendo con el ejemplo de los gastos en gasolina, Gemini deberá buscar nombres de gasolineras reales en el concepto de la transacción.
- El esquema completo de la tabla `transaccion`.
- Aviso sobre el uso del valor absoluto en consultas que requieran ordenar las transacciones, como por ejemplo la categoría en la que más se gasta.
- En consultas sobre comparaciones, Gemini deberá devolver un valor por fila en vez de resumirlo devolviendo únicamente el mayor o menor valor. Esto permitirá generar gráficos de mayor calidad.

6.2. Seguridad: validación y permisos

El agente utiliza los siguientes mecanismos de seguridad una vez ha traducido la pregunta del usuario de NL a SQL:

- **Validación por lista blanca**: solo se ejecuta si la consulta empieza por SELECT y contiene una única sentencia.
- **Conexión de solo lectura**: aunque la validación fallara, el motor rechaza cualquier escritura.
- **Acceso a tabla transaccion**: se comprueba si aparece `from transaccion` en la consulta, y en caso negativo, se redirecciona la conversación a la rama de Charla.

Los dos primeros mecanismos de seguridad son imprescindibles para evitar ataques de inyección SQL, mientras que el último mecanismo intenta mitigar los errores provocados por el no determinismo de las llamadas a Gemini.

7. Operaciones financieras

7.1. Invocación de funciones

Las funcionalidades para consultar el saldo de la cuenta y enviar un Bizum difieren de una llamada normal a Gemini, y utilizan una herramienta llamada **tool calling**.

El **tool calling** es un mecanismo que permite al modelo indicar qué función debe ejecutarse y con qué argumentos, en lugar de responder únicamente con texto. Para ello se le proporciona una **declaración** de cada función disponible: su nombre, una descripción de para qué sirve y los parámetros que espera, con su tipo y significado.

Es importante señalar que estas declaraciones no contienen ninguna lógica: describen las funciones, pero no las implementan. Las funciones reales residen en el módulo `database.py` y es el propio agente quien las ejecuta. El modelo se limita a decidir cuál invocar y a extraer los argumentos del mensaje del usuario, de modo que el control sobre lo que realmente ocurre permanece en el código.

7.2. Consulta de saldo

Esta función es la más sencilla, ya que no requiere ningún parámetro para ser invocada. Como se mencionó anteriormente en el diseño de la base de datos, la función únicamente realizará una suma de los importes de todas las transacciones.

Al tratarse siempre de la misma consulta, esta se encuentra escrita directamente en el código en lugar de generarse mediante el traductor NL→SQL. El modelo únicamente decide cuándo invocar la función, no cómo consultarla. Esto evita una llamada adicional a Gemini y, sobre todo, elimina cualquier posibilidad de error: la consulta del saldo es determinista, mientras que una generada por el modelo podría no serlo.

7.3. Transferencias vía Bizum

Enviar un Bizum ya no es una simple consulta a la base de datos, sino que es una operación que la modifica. Por tanto, es necesario abrir una nueva conexión de escritura para añadir la nueva transacción. Este detalle es importante, ya que el resto del sistema opera en solo lectura, así que al ser la única operación de escritura, se intenta que esta conexión esté abierta el menor tiempo posible debido a que es la más vulnerable.

Además, se realizarán varias validaciones en código: el importe del Bizum debe ser un valor positivo mayor que cero y no puede superar el saldo total de la cuenta.

Respecto a los parámetros de la función, se requerirá que el usuario especifique el destinatario y la cantidad del Bizum antes de poder finalizar la operación. Se comprobará en código que los parámetros existan y sean del tipo correcto, ya que el modelo podría devolver la cantidad como texto u omitir un campo.

7.4. Confirmación de operaciones

Al tratarse de una operación que mueve dinero, el envío de un Bizum no se ejecuta de forma inmediata. Cuando el modelo indica que debe invocarse la función `bizum`, el agente no la ejecuta: guarda los argumentos extraídos, muestra al usuario el importe y el destinatario, y espera su respuesta. Solo si el usuario confirma se realiza la transferencia.

El esquema resultante es por tanto: el modelo propone la operación, el código la guarda, el usuario decide y el código la ejecuta. La confirmación se convierte en una garantía del sistema.

La respuesta del usuario se interpreta mediante una llamada a Gemini que la clasifica como afirmativa, negativa o ambigua. En este último caso la operación se cancela por seguridad, siguiendo el criterio habitual de las aplicaciones de banca: ante una respuesta que no se entiende con claridad, es preferible cancelar y que el usuario repita la petición.

8. Respuestas visuales

8.1. Decisión del tipo de gráfico

Tal y como se indica en el reto, el tipo de gráfico que se va a mostrar es decidido por el agente en tiempo real dependiendo de los datos devueltos por la consulta. Para conseguir esto, se realiza una llamada a Gemini con un prompt que contiene la pregunta del usuario, la consulta SQL, el resultado de la consulta y los nombres de cada columna del resultado. Esta llamada devuelve un JSON con el tipo de gráfico, qué columna va de etiquetas, cuál de valores y un título para el gráfico.

El modelo estará obligado a seleccionar entre una lista cerrada de tipos de gráficos (`none`, `pie`, `bar`, `line`), ya que son los únicos tipos de gráficos que se pueden crear. En el caso de `none`, esto significará que Gemini ha decidido que no es necesario mostrar un gráfico, ya sea porque el resultado de la consulta es nulo, contiene una única fila, etc.

El formato de la salida de Gemini será JSON utilizando `response_mime_type: "application/json"`, ya que mencionarlo en el prompt no siempre asegura que se genere un formato JSON.

8.2. Construcción de la visualización

Una vez que el modelo ha decidido el tipo de gráfico y tenemos toda la información del gráfico en formato JSON, el módulo `charts.py` se encargará de generar el gráfico. Concretamente la función `build_plot`, que realizará las siguientes tareas: localizar las columnas por nombre utilizando la información del JSON, extraer las dos listas de datos, aplicar valor absoluto a los datos (ya que el usuario espera ver cantidades positivas), y finalmente llamar a Plotly según el tipo de gráfico decidido por Gemini y devolverlo.

8.3. Integración en la conversación

Gradio permite que la función del agente devuelva componentes además de texto, por lo que el gráfico no se muestra en un panel independiente, sino dentro de la propia conversación, justo debajo de la respuesta redactada. De este modo el usuario recibe el dato y su representación visual como parte de la misma respuesta.

La decisión de generar un gráfico implica una llamada adicional a Gemini, por lo que se ha añadido una comprobación previa: si el resultado de la consulta contiene una única fila, no se pregunta al modelo, ya que con un solo valor no existe nada que representar. Esta comprobación evita una llamada innecesaria en la mayoría de consultas.

9. Estudio de rendimiento

9.1. Metodología de medición

Para medir la latencia se ha instrumentado la función principal del agente, registrando el tiempo transcurrido desde que se recibe el mensaje del usuario hasta que se devuelve la respuesta completa. De este modo se mide el tiempo que percibe el usuario, incluyendo todas las llamadas a Gemini, el acceso a la base de datos y la construcción del gráfico cuando procede.

Se han definido cinco escenarios, uno por cada tipo de interacción que soporta el sistema, y cada uno se ha ejecutado cinco veces con la misma pregunta. Las repeticiones se han intercalado entre escenarios en lugar de ejecutarse de forma consecutiva, para evitar que todas las medidas de una misma categoría coincidan con el mismo estado de la red.

Junto al tiempo se indica el número de llamadas a Gemini que realiza cada escenario, ya que es el factor que determina el resultado. Todas las llamadas son secuenciales: cada una necesita el resultado de la anterior, por lo que sus tiempos se acumulan.

9.2. Resultados

La Tabla 1 recoge los tiempos obtenidos.

Escenario	**Llamadas**	**Media (s)**	**Mediana (s)**
Charla	2	2,27	1,96
Consulta de saldo	2	2,09	1,86
Bizum (petición)	2	1,47	1,48
Bizum (confirmación)	1	0,86	0,85
Consulta sin gráfico	3	2,93	2,97
Consulta con gráfico	4	4,26	3,54

Tabla 1: Latencia por escenario. Cinco ejecuciones de cada uno.

Se incluyen media y mediana porque algunas ejecuciones presentaron picos aislados de latencia probablemente debido a la red o a la carga de los servidores del modelo, y no al comportamiento del sistema. La mediana refleja mejor el caso habitual, mientras que la diferencia entre ambas indica la dispersión de las medidas.

Es evidente que **la latencia tiene una correlación directa con el número de llamadas** encadenadas a Gemini. Este dato se puede observar claramente si comparamos los tiempos de una Consulta de saldo (2 llamadas) y una Consulta sin gráfico (3 llamadas), aumentando en el segundo caso alrededor de un segundo el tiempo de la Consulta de saldo. Con lo mencionado antes y observando el tiempo que tarda una Confirmación de Bizum (1 llamada), podemos estimar el tiempo de cada llamada a Gemini en el rango de 0.8 a 1.0 segundos. Al conocer el tiempo medio de cada llamada a Gemini, se podría predecir cómo escala el sistema si se quieren añadir funcionalidades nuevas únicamente conociendo el número de llamadas que realizan.

9.3. Optimizaciones aplicadas

La primera optimización parte de la observación anterior: si la latencia depende del número de llamadas, la vía para reducirla es evitar las que no aportan nada. La llamada que decide el gráfico solo tiene sentido cuando existe algo que representar, por lo que se comprueba previamente el número de filas devueltas por la consulta: si es una sola, no se pregunta al modelo y se omite la llamada. Esta comprobación afecta a la mayoría de las consultas, que devuelven un único valor, y supone un ahorro cercano a medio segundo.

La segunda no reduce el tiempo real, sino el percibido. En lugar de esperar a tener la respuesta completa, el agente muestra mensajes de progreso conforme avanza por el flujo: al identificar la intención, al consultar la base de datos y al analizar los resultados. El usuario recibe así una primera respuesta en menos de un segundo, en lugar de esperar varios segundos ante una pantalla sin actividad. El tiempo total no varía, pero la sensación de agilidad sí, y además informa superficialmente de qué está haciendo el sistema en cada momento.

10. Evaluación del sistema

10.1. Metodología

Para evaluar la calidad de la traducción de lenguaje natural a SQL se ha definido un conjunto de 25 preguntas agrupadas en siete bloques, cubriendo los distintos tipos de consulta que el sistema debe soportar: agregados simples, filtros temporales, consultas por categoría, consultas por concepto o comercio, comparativas, evolución temporal y casos límite.

Para cada pregunta se ha registrado la consulta SQL generada y la respuesta final mostrada al usuario, evaluando de forma separada si la consulta refleja correctamente la intención del usuario y si la respuesta resulta adecuada.

10.2. Resultados

#	Pregunta	SQL	Respuesta
1	¿Cuánto gasté en 2025?	✓	✓
2	¿Cuál es mi saldo?	—	✓
3	¿Cuánto ingresé en total en 2025?	✓	✓
4	¿Cuánto gasté en abril de 2025?	✓	✓
5	¿Cuánto gasté el año pasado en el primer trimestre?	✓	✓
6	¿Cuál fue mi último movimiento?	✓	✓
7	¿Cuánto gasté en Restaurantes en 2025?	✓	✓
8	¿En qué categoría gasto más?	✓	✓
9	¿Cuánto gasté en cada categoría en 2025?	✓	✓
10	¿Cuánto he gastado en Salud este año?	✓	✓
11	¿Cuánto gasté en Mercadona en 2025?	✓	✓
12	¿Cuánto gasté en gasolina en 2025?	✓	✓
13	¿Qué suscripciones tengo?	×	×
14	¿Cuánto pago de hipoteca?	✓	✓
15	¿Cuánto gasté en McDonald's?	✓	✓
16	¿Gasté más en junio o en julio de 2025?	✓	✓
17	¿Cuánto gasté en Ocio en 2025?	✓	✓
18	¿Y en 2026?	✓	✓
19	¿Cómo evolucionaron mis gastos en 2025?	✓	✓
20	¿Cuánto gasté cada mes en Supers e Hipers en 2025?	✓	✓
21	¿Cuánto gasté en el seguro del coche?	✓	✓
22	¿Cuánto gasté en Zara en enero de 2025?	✓	✓
23	¿Cuánto gasté en criptomonedas?	✓	✓
24	¿Cuál fue mi mayor gasto en 2025?	×	×
25	¿Cuántas transacciones tengo registradas?	✓	✓

Tabla 2: Resultado de la evaluación. La consulta 2 se resuelve mediante invocación de función y no genera SQL.

La Tabla 2 recoge el resultado de las 25 consultas evaluadas. Se distingue entre la corrección de la consulta SQL generada y la de la respuesta final mostrada al usuario, ya que son aspectos independientes: una consulta correcta puede dar lugar a una respuesta incompleta.

10.3. Análisis de los fallos

En la tabla de resultados se pueden observar los siguientes dos fallos:

- **Pregunta 13**. Esta pregunta va sobre las suscripciones activas. El traductor SQL (llamada a Gemini) interpreta por suscripciones marcas como HBO o Amazon Prime, lo cual es correcto, pero se deja fuera suscripciones que aparecen en la base de datos como DigiMobil o Sanitas. Esto ocurre principalmente porque `SQL_PROMPT` da ejemplos de streaming, así que el modelo busca esa lista. Una consulta de suscripciones debería detectar además pagos recurrentes (mismo concepto repetido cada mes), no comparar contra marcas conocidas, aunque ni así estaría asegurado que el agente responda correctamente. Un ejemplo de compra recurrente cada mes que no es suscripción sería una compra mensual del mismo producto, lo que se reflejaría en la cuenta como un pago recurrente con mismo importe sin ser una suscripción.
- **Pregunta 24**. Esta pregunta va sobre el mayor gasto de 2025. El problema es que el agente responde correctamente con el importe del mayor gasto de 2025, sin embargo no indica ninguna información adicional sobre la transacción a la que pertenece. Además, se realizó una prueba y se volvió a preguntar la misma pregunta al agente, y en este caso sí respondió con el concepto de la transacción además del importe, por lo que este es un claro ejemplo del no determinismo de las llamadas a un LLM.

Ambos fallos comparten una característica: en ningún caso se produce un error de sintaxis SQL ni de ejecución. Las consultas generadas son válidas y se ejecutan sin problema, lo que falla es la interpretación de la intención del usuario. Este tipo de error resulta difícil de detectar, ya que la respuesta obtenida es coherente y suena correcta, y solo contrastando con los datos reales se aprecia que está incompleta.

Por tanto, podemos obtener dos conclusiones. La primera es que evaluar un sistema de este tipo no puede limitarse a comprobar que no se producen errores, sino que se debe verificar los resultados con los datos. La segunda es que la tasa de acierto obtenida, aunque elevada, no es una garantía: al tratarse de un componente no determinista, la misma pregunta puede resolverse correctamente en una ejecución e incorrectamente en la siguiente, tal y como se ha observado en la pregunta 24.

11. Manual de uso

11.1. Requisitos e instalación

El proyecto requiere Python 3.12 y una clave de la API de Gemini, que puede obtenerse de forma gratuita en Google AI Studio. El código está disponible en el siguiente repositorio: https://github.com/roberto20961/habla-con-tu-dinero.

Los pasos de instalación son los siguientes:

```
git clone https://github.com/roberto20961/habla-con-tu-dinero.git
cd habla-con-tu-dinero

python3 -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt
```

A continuación debe crearse un archivo `.env` en la raíz del proyecto con la clave de la API:

```
GEMINI_API_KEY=tu_clave_aqui
```

11.2. Ejecución

La base de datos no se incluye en el repositorio, ya que se genera mediante el propio modelo. Antes de la primera ejecución es necesario crearla:

```
python crear_db.py
```

Este script solo debe ejecutarse una vez. Volver a ejecutarlo sustituye por completo los datos existentes por un conjunto nuevo.

Por último, el asistente se lanza desde el directorio `src`, ya que las rutas a la base de datos son relativas:

```
cd src
python app.py
```

La interfaz queda accesible en `http://127.0.0.1:7860`.

12. Conclusiones

Tras realizar las pruebas, tanto de rendimiento como de precisión, y de probar repetidamente todas las funcionalidades durante el desarrollo del agente, puedo expresar que el resultado es muy satisfactorio.

No solo ha sido construir un chatbot funcional desde cero, sino que he aprendido el proceso de construir un wrapper sobre un LLM y darle cualquier funcionalidad, cosa que sería imposible conseguir utilizando métodos clásicos como los sistemas basados en reglas. Y más que el código en sí, me parece más interesante el pipeline del mensaje y cómo se tiene que procesar en cada parte del agente, ya que entendiendo esa idea se puede construir un chatbot con la misma filosofía para cualquier campo o empresa.

Al tratarse de un reto sobre IA Responsable, pienso que no había mejor ejemplo que el asistente de un sistema bancario, ya que estos requieren mucha seguridad y control. Por eso mismo, en el proyecto hay bastantes métodos de seguridad y validación antes de ejecutar cualquier consulta en la base de datos o de realizar un nuevo movimiento en la cuenta, y más conociendo el no determinismo de los LLM.

Respecto a líneas futuras del proyecto, el principal enfoque debería ir orientado hacia la optimización del agente mediante la reducción de llamadas a Gemini. Una forma de conseguirlo sería fusionando diferentes prompts, lo que resultaría en un menor número de llamadas, y por tanto, en una menor latencia.

Finalmente, considero que una frase resume adecuadamente la filosofía seguida durante el desarrollo del agente: **el modelo aporta la capacidad de generalización, mientras que el código se encarga de proporcionar las garantías**, verificando, validando y controlando aquello que genera el LLM antes de permitir que tenga un efecto sobre el sistema.